Design of a High-Speed 16×16 Signed Booth-Dadda Tree Multiplier

Nagendra Kudva
School Of Electrical Engineering, Manipal Institute of Technology
Manipal, India
Reg. No: 240907002

Pramit Karmakar
School Of Electrical Engineering, Manipal Institute of Technology
Manipal, India
Reg. No: 240907726

Suprit Shanbhag
School Of Electrical Engineering, Manipal Institute of Technology
Manipal, India
Reg. No: 240907774

***Abstract*—This report presents the complete structural design and Boolean derivation of a 16×16 radix-4 Booth-Dadda multiplier. The architecture combines a radix-4 Booth encoder to reduce partial products, a Modified Sign Extension (MSE) scheme to optimize area, a Dadda reduction tree for parallel matrix compression, and a final 32-bit carry-propagate addition stage. The design processes 16-bit signed operands, generating eight partial-product rows. The Dadda tree reduces the peak column height of 9 down to 2 across four compression stages using 119 combinational compressors. The structural implementation emphasizes primitive gate derivation and rigorous routing.**

***Index Terms*—Radix-4 Booth Encoding, Dadda Tree, Combinational Multiplier, Modified Sign Extension, VLSI Arithmetic, Logisim.**

I. Introduction

Multiplication is a fundamental and computationally intensive operation in digital signal processing (DSP), arithmetic logic units (ALUs), and high-performance microprocessors. As the demand for high-speed and low-power very-large-scale integration (VLSI) systems grows, the algorithmic efficiency of multiplier architectures becomes a critical performance bottleneck. The multiplication process strictly consists of three primary stages: partial product generation, partial product reduction, and final addition. The integrated architecture described in this report optimizes all three stages to minimize critical path delay, restrict dynamic power dissipation, and heavily optimize spatial hardware utilization compared to standard array multipliers.

II. Evolution of Multiplier Architectures

Early arithmetic logic units relied on standard shift-and-add algorithms or array multipliers to execute binary multiplication. While array multipliers possess a highly regular geometric structure suitable for straightforward VLSI layout, their worst-case propagation delay scales linearly with the operand bit-width ($O(N)$). To break this linear delay dependency, parallel tree multipliers were introduced to compress partial products concurrently rather than sequentially. The Wallace tree multiplier, introduced in 1964, utilizes parallel arrays of full and half adders to compress bit columns as early as mathematically possible [1]. This achieves a logarithmic delay profile ($O(\log N)$) but results in a highly irregular wiring layout and a sub-optimal total gate count because compression is forced even when it does not reduce the overall depth of the stage. In 1965, L. Dadda improved upon this methodology by introducing a mathematically optimal reduction sequence [2]. The Dadda algorithm dictates that columns should only be compressed when their height exceeds a strict threshold sequence (e.g., $9 \rightarrow 6 \rightarrow 4 \rightarrow 3 \rightarrow 2$). By delaying compression until absolutely necessary, Dadda multipliers require significantly fewer 3:2 and 2:2 compressors in the combinational tree compared to Wallace designs. While this requires a slightly wider final carry-propagate adder, the severe reduction in active gate area directly lowers spatial overhead and limits spurious switching power.

Beyond tree reduction, the generation of the partial products themselves represents a major architectural bottleneck. Standard multipliers generate exactly N partial product rows for an N-bit multiplier. To alleviate this, A. D. Booth introduced a recoding algorithm [3], which was later modified by O. L. MacSorley into Radix-4 (Modified) Booth Encoding [4]. Rather than operating bit-by-bit, Radix-4 Booth encoding scans the multiplier operand in overlapping three-bit triplets, mapping them to specific operations on the multiplicand (0, +1X, -1X, +2X, -2X). This recoding structurally halves the number of partial products, shrinking the initial matrix depth before any tree compression is even required.

In modern VLSI, the focus has shifted toward combining these optimal stages, Booth encoding for generation and Dadda trees for reduction, while addressing the overhead of two's complement signed arithmetic. In a naive signed Booth array, sign-extending negative partial products out to the full $2N$ bit-width creates a massive, resource-wasting triangle of redundant sign bits. Contemporary implementations bypass this entirely using a Modified Sign Extension (MSE) method [5]-[6]. By inverting the most significant bit (MSB) of each row and injecting a precise pattern of constant '1's, the architecture mathematically contains the sign bits without widening the datapath. This report details the structural implementation of these blocks, their theoretical derivations, and the observed simulation results across critical corner cases.

III. Objectives

The primary objectives of this project are as follows:

- To design and verify a high-speed 16×16-bit signed multiplier architecture.

- To optimize the partial product generation stage by implementing Radix-4 Booth encoding to minimize the initial depth of the matrix.
- To implement a Modified Sign Extension (MSE) algorithm to eliminate redundant sign-extension bits and restrict the peak column height.
- To construct a combinational Dadda reduction tree that mathematically compresses the partial product matrix using the minimum necessary spatial hardware.
- To synthesize and simulate the complete architecture at the primitive gate level to validate functional correctness against standard multiplier benchmarks.

IV. Methodology and Simulation Tools

The methodology for this project follows a strict structural design flow, transitioning from theoretical Boolean logic derivation to gate-level instantiation and system-level integration.

A. Simulation Environment

The entire architecture was implemented and simulated using **Logisim**, a digital logic simulator [1]. Logisim was selected as the primary platform to allow for explicit, gate-level visualization of the combinational logic and direct observation of signal propagation throughout the compression matrix.

B. Design Approach

The design was executed using a bottom-up hierarchical methodology:

1) **Primitive Gate Design:** The fundamental building blocks (Booth encoders, selectors, Half/Full Adders) were derived using standard Boolean logic and implemented using basic NOT, AND, OR, and XOR gates.
2) **Sub-module Instantiation:** Primitive gates were packaged into intermediate modules, such as the 17-bit Booth Row Generator.
3) **Top-Level Integration:** Parallel Row Generators were instantiated in the top-level schematic and aligned into a global matrix, optimized by the MSE logic.
4) **Reduction and Final Addition:** A multi-stage Dadda reduction tree was routed using labeled tunnels to compress the matrix into two 32-bit vectors, culminating in a final Carry-Propagate Addition stage.

V. Design Implementation

A. Radix-4 Booth Encoding and Generation

The partial product generation stage leverages radix-4 Booth recoding, scanning the 16-bit multiplier Y two bits at a time using an overlapping 3-bit window: Y_up (next-higher bit), Y (current bit) and Y_down (previous bit).

1) Operation Mapping: Each window selects one of five operations on the multiplicand X: 0, +1X, -1X, +2X, or -2X. The encoder translates these into three control signals: Neg (sign), X1 (select $\pm$1X), and X2 (select $\pm$2X).

2) Boolean Derivation of Booth Encoded Logic:

Table I. Complete radix-4 Booth window $\rightarrow$ operation $\rightarrow$ (Neg, X1, X2) mapping

Window	Operation	Neg	X1/X2
0 0 0	0	0	0/0
0 0 1	+1 $\times$X	0	1/0
0 1 0	+1$\times$X	0	1/0
0 1 1	+2$\times$X	0	0/1
1 0 0	-2$\times$X	1	0/1
1 0 1	-1 $\times$X	1	1/0
1 1 0	-1 $\times$X	1	1/0
1 1 1	0	1	0/0

Figure 1. Karnaugh maps for the Booth control signals X1 and X2

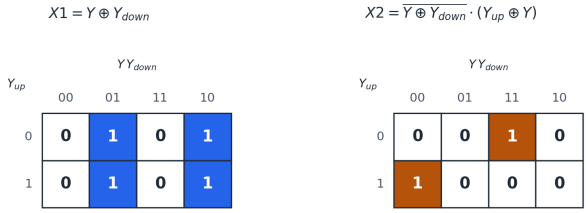


Fig. 1. Karnaugh maps for X1 and X2 over (Y_up, Y, Y_down); X2 isolates exactly minterms 011 and 100.

a) Neg: The sign of the operation is fixed entirely by the window's most-significant bit.

$$Neg = Y_{up}$$

Therefore, no physical gate is needed in the Logisim schematic; Neg is a direct wiretap from the Y_{up} input pin. This is confirmed by the netlist, where Y_{up} feeds both the Neg output and the internal XOR/AND logic with no intervening component.

b) X1 Single-Multiplicand Select: +1X and $-$1X occur whenever Y and Y_{down} differ (windows 001, 010, 101, 110), representing a direct XOR relationship:

$$X1 = Y \oplus Y_{down}$$

This matches the single XOR-gate implementation in the Booth encoder block.

c) X2 Double-Multiplicand Select: +2X and $-$2X occur only at windows 011 and 100, which are the two minterms where Y and Y_{down} agree but disagree with Y_{up}:

$$X2 = (Y \oplus Y_{down})\prime \cdot (Y_{up} \oplus Y)$$

The Logisim netlist implements this as a sum of the two minterms directly, rather than instantiating complex XOR/XNOR gates. Three NOT gates produce the complements of Y_up, Y, and Y_down, two 3-input AND gates form the 011 and 100 minterms, and a 2-input OR gate combines them into X2.

3) Boolean Derivation of 1-Bit Selector: The 1-Bit Selector produces a single partial product bit by evaluating the corresponding bits of the multiplicand. These inputs include the current bit of X, the previous bit of X (to account for potential shifts), and the three shared control signals (Neg, X1, X2) distributed by the row's corresponding encoder.

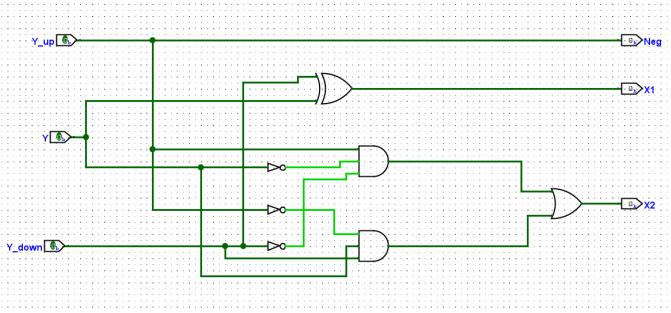


Fig. 2. Booth encoder signal path, represented in standard logic gates.

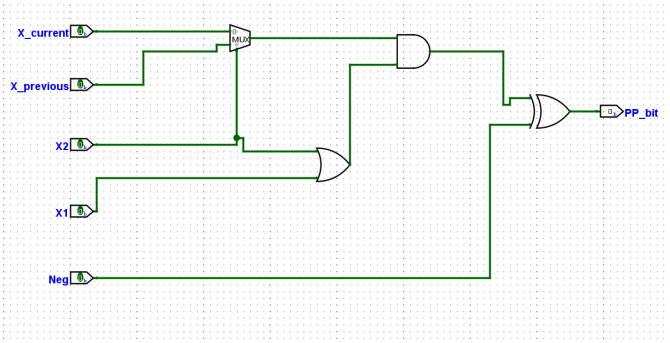


Fig. 3. 1-bit selector: MUX → AND/OR zero-mask → XOR sign-flip, matching the four gates in the Booth_selector_1bit component.

a) Shift Select M: A 2:1 multiplexer, selected by X2, chooses between the unshifted and the left-shifted tap of X:

$$M = (X_{current} \cdot X2\prime) + (X_{previous} \cdot X2)$$

b) Zero Masking S: When neither X1 nor X2 is asserted (the encoded digit is 0), the output bit must be forced low regardless of the multiplexer selection:

$$S = M \cdot (X1 + X2)$$

c) Sign Flip OUTPUT: A final XOR against the Neg signal implements the one's-complement half of two's-complement negation:

$$Output = S \oplus Neg$$

4) Structural Composition of Row Generator: The row generator comprises one Booth encoder wired to 17 parallel 1-Bit Selectors, all sharing the encoder's Neg/X1/X2 bus. Each successive selector's X_previous input taps X one bit index higher than the selector before it, implementing the necessary two-bit left shift between Booth groups.

B. Sign Extension, Dadda Reduction, Final Addition

1) Data Ingestion and the Eight Booth Rows: The top-level circuit takes two 16-bit inputs, X and Y. Y is split into eight overlapping 3-bit Booth windows (window 0 pads Y_{down} with a hardcoded Constant 0, since there is no bit -1). Eight parallel Booth row generator instances evaluate these windows against X, each producing a 17-bit raw partial product PP_r and a 1-bit negation/correction signal N_r. This yields the top-level inventory: 8 × Booth_row_generator, 8 × window-extraction splitters, and a Constant 0 feeding the first window's Y_down.

2) Modified Sign-Extension (MSE): Naively sign-extending every row to the full 32-bit product width would build a deep, redundant triangle of repeated sign bits, generating significant delay and gate overhead. Instead, the design employs the standard Modified Sign Extension technique for radix-4 Booth arrays. Each row's natural sign bit is preserved, and two small correction dots are logically injected just past it.

For rows 0 through 6, the row's MSB is inverted and placed at relative bit 17, and a constant 1 is placed at relative bit 18, immediately to its left. Row 7 (the final row) requires only the inverted-MSB bit, with no trailing constant. A single standalone rollover constant 1 is wired into column 17 of the global matrix to absorb the ripple-carry deficit left by the logic reduction. Finally, each row's own N_r negation-correction bit drops into the least-significant column of that row's shifted position.

3) Dadda Reduction Tree:

a) Matrix Initialization and Reduction Sequence: The Dadda reduction algorithm minimizes the number of compression stages required to flatten a matrix by strictly targeting a mathematically defined sequence of maximum column heights. In this architecture, the radix-4 Booth shifting, combined with the MSE correction bits and the standalone rollover constant, generates an initial global matrix with a peak column height of nine bits. To compress this peak down to the two rows required for standard addition, the Dadda methodology dictates a strict four-stage height reduction sequence: $9 \rightarrow 6 \rightarrow 4 \rightarrow 3 \rightarrow 2$.

b) Combinational Compression Mechanism: To achieve each intermediate target height, the algorithm applies a parallel layer of combinational compressors exclusively to the specific bit-columns that exceed the stage's height limit. This compression is executed using Full Adders, which function as 3:2 counters by consuming three bits from a column to output two bits, and Half Adders, which function as 2:2 counters. During physical routing, every resulting sum bit drops vertically into its original column index, while every carry bit routes diagonally one column to the left to propagate into the subsequent compression stage.

c) Stage-by-Stage Hardware Allocation: The complete reduction of the 16×16 matrix utilizes exactly 119 discrete compressors, comprising 97 Full Adders and 22 Half Adders. The first stage targets the densest central columns, utilizing 17 Full Adders and 10 Half Adders to compress the initial peak height of 9 down to 6. The second stage deploys 32 Full Adders and 6 Half Adders to level the resulting expanded plateau, achieving a uniform maximum height of 4. The third stage applies 22 Full Adders and 3 Half Adders to perform a flat ripple reduction, bringing the matrix height to 3. The fourth and final compression stage utilizes 26 Full Adders and 3 Half Adders to sweep across the remaining columns, successfully yielding exactly two remaining wires, a Sum vector and a Carry vector, across the entire 32-bit span.

d) Structural Implementation and Routing: Within the schematic environment, each Full Adder is instantiated as a

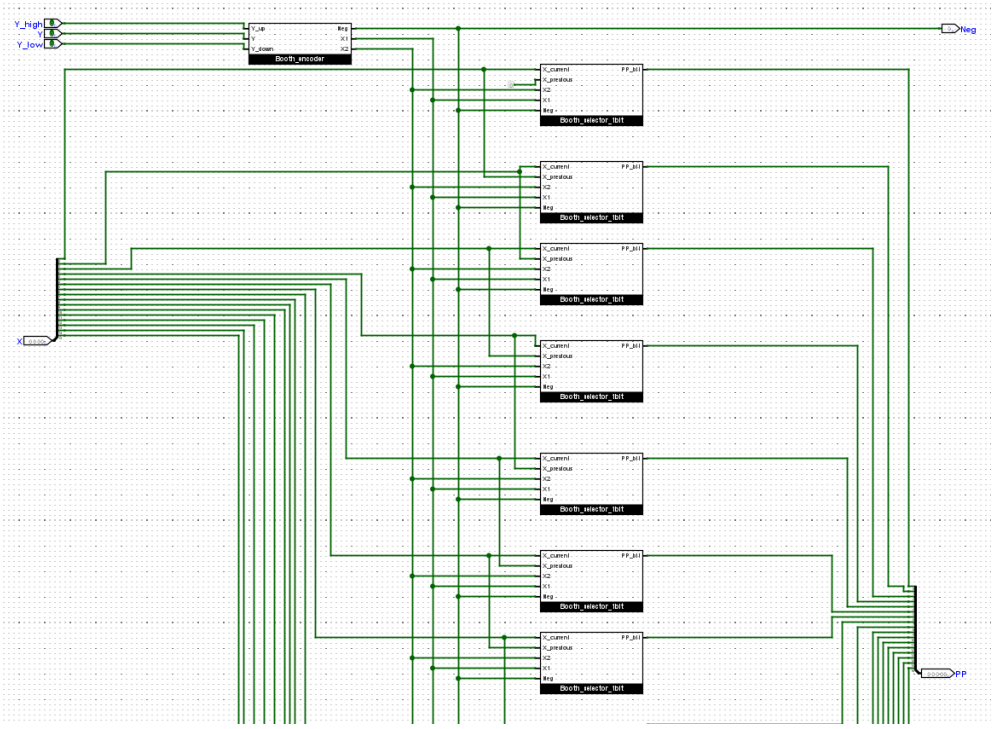


Fig. 4. One shared control bus (Neg, X1, X2) fans out to 17 selectors, each tapping X at a successively shifted bit index.

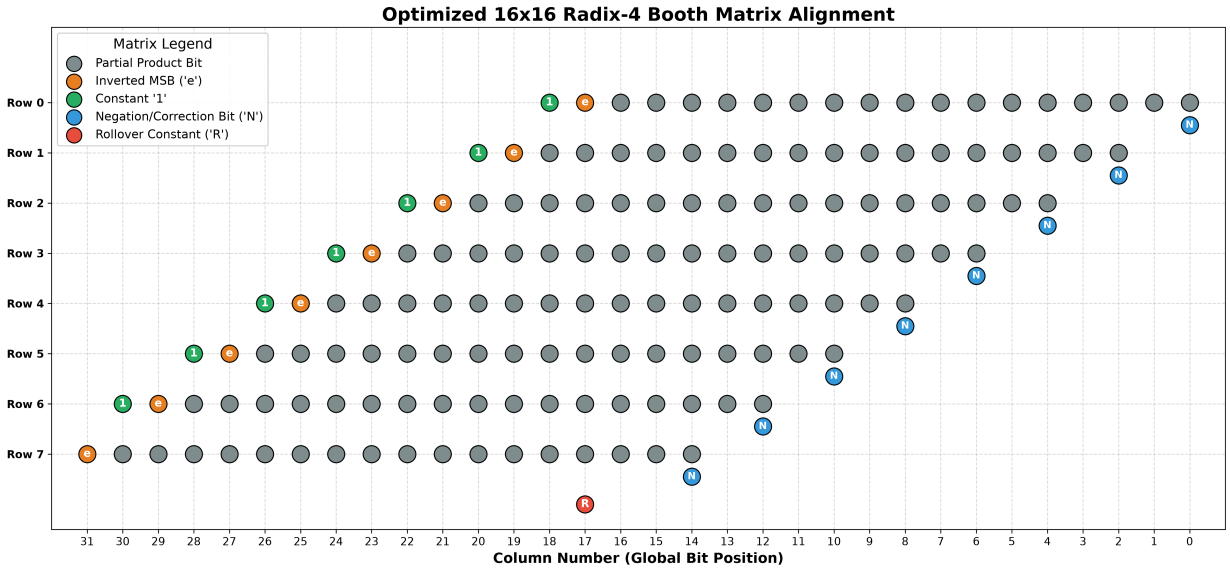


Fig. 5. Initial uncompressed dot matrix representation.

structural subcircuit built from two Half Adders and an OR gate. To manage the immense routing complexity between the four Dadda stages, every intermediate sum and carry wire is routed through uniquely named tunnels. This requires 667 distinct tunnel connections across the canvas to maintain a visually traceable, column-labeled layout without crossed wires. While this rigorous structural approach maximizes the underlying primitive gate footprint, it guarantees mathematically strict adherence to the Dadda compression sequence before the vectors are passed to the final adder.

4) Final Carry-Propagate Addition: Following the reduction tree, every column holds at most 2 bits. These are gathered into two distinct 32-bit vectors by two 32-wide merging splitters (a Sum vector and a Carry vector). Any column that produced only a single bit is padded with a Constant 0 to ensure neither vector contains a floating state. To execute the final vector merge, the design utilizes the native 32-bit `Adder` component provided within Logisim's arithmetic library. Utilizing this built-

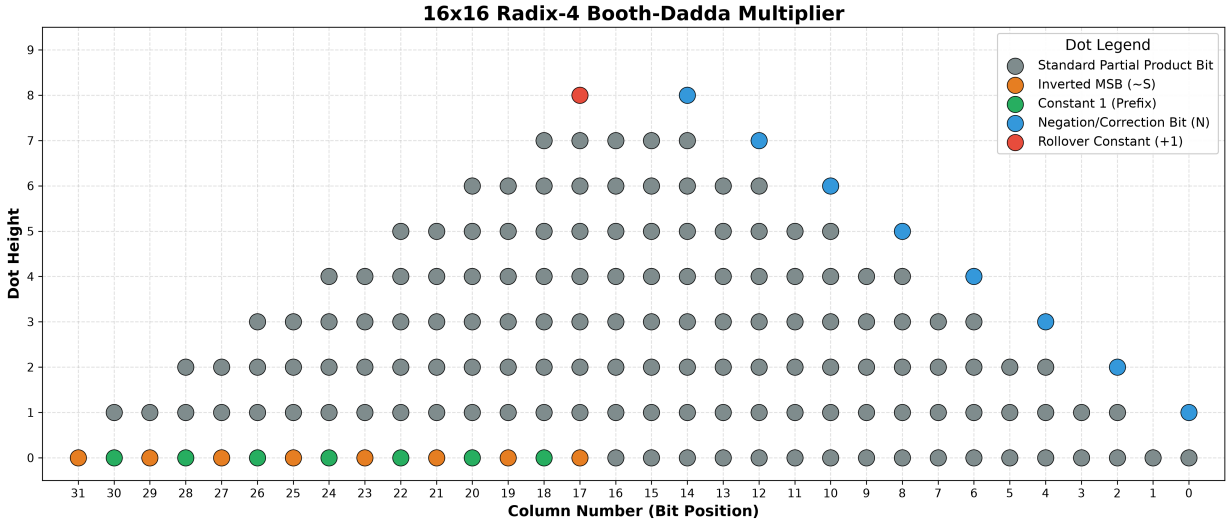


Fig. 6. Per-row MSE dot pattern, modeled from the netlist and structurally cross-checked against the component inventory.

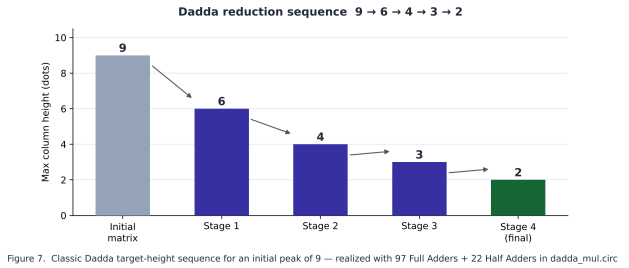


Figure 7. Classic Dadda target-height sequence for an initial peak of 9 — realized with 97 Full Adders + 22 Half Adders in dadda_mul.circ

Fig. 7. Target-height compression sequence for an initial peak height of 9.

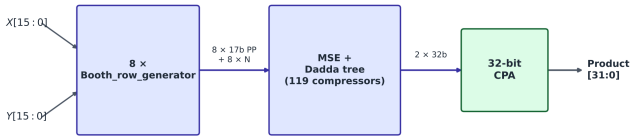


Figure 9. Complete 16×16 radix-4 Booth–Dadda multiplier — fully built in dadda_mul.circ

Fig. 8. Complete 16×16 radix-4 Booth–Dadda multiplier schematic.

in component functions as the final Carry-Propagate Adder (CPA), accurately producing the finalized 32-bit product while abstracting the final carry chain to maintain the primary design focus on the optimization of the combinational Dadda tree.

VI. Results and Discussion

A. Functional Verification and Corner Cases

The primary objective of the functional verification phase is to prove the integrated architecture computes signed two's complement arithmetic correctly across all numerical boundaries. In the Logisim simulation environment, this was validated by probing the inputs and outputs of the top-level `TEST2` wrapper circuit.

To ensure mathematical robustness, the multiplier was subjected to extreme corner cases. These values typically induce overflow, incorrect sign evaluation, or cascading logic failures in poorly optimized arrays (e.g., multiplying two maximum negative numbers). As demonstrated in Table II and visually confirmed in Fig. 9, the Booth-Dadda architecture successfully computed the correct 32-bit signed product for all critical boundaries without anomalous floating states or contention errors. Fig. 9 explicitly illustrates the successful evaluation of the Max Negative × Max Negative corner case, representing the most hardware-intensive logic path.

Table II. Functional Verification of Corner Cases

Test Case Description	X (Hex)	Y (Hex)	Expected Product	Result
Zero Crossing	0x0000	0x7FFF	0x00000000	PASS
Max Pos × Max Pos	0x7FFF	0x7FFF	0x3FFF0001	PASS
Max Neg × Max Neg	0x8000	0x8000	0x40000000	PASS
Max Pos × Max Neg	0x7FFF	0x8000	0xC0008000	PASS
Negative Identity	0xFFFF	0x0001	0xFFFFFFFF	PASS
Random Mixed-Sign	0xFF83	0x002D	0xFFFFE9F7	PASS

B. Comparative Performance Analysis

To quantify the efficiency of the implemented design, its theoretical performance is evaluated against traditional 16×16 multiplier architectures.

As demonstrated in Table III, a standard array multiplier requires processing 16 distinct rows sequentially, yielding an $O(N)$ combinational delay across 15 logic stages. While standard Wallace or Dadda Trees reduce this depth logarithmically to 6 stages, they still require the full generation of 16 partial product rows.

By implementing Radix-4 Booth encoding, the architecture instantly halves the generated rows to 8. This restricts the initial matrix peak height to 9 bits, allowing both standard Wallace and Dadda algorithms to flatten the tree in just 4 parallel stages. However, as noted in the architectural comparison, while Booth-Wallace and Booth-Dadda achieve the identical

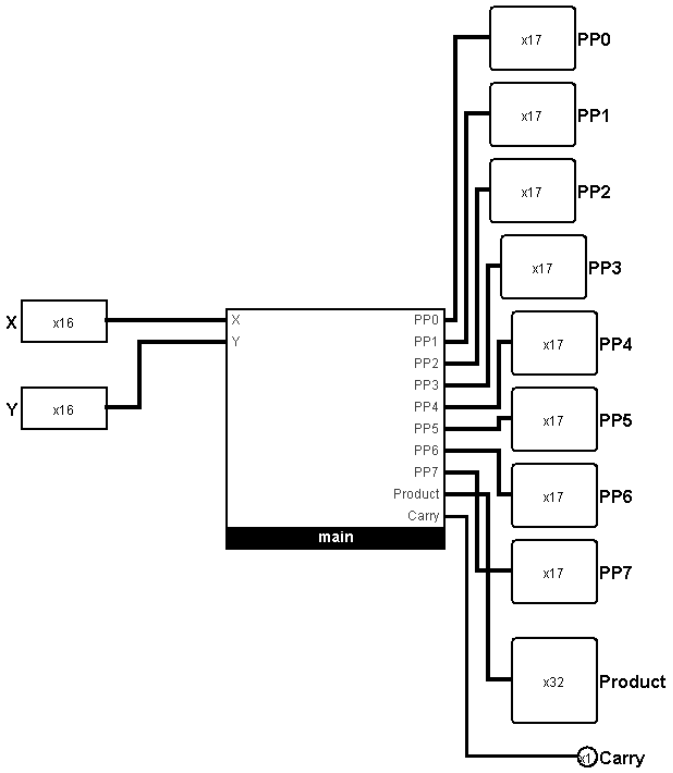


Fig. 9. Logisim simulation of the `TEST2` wrapper circuit verifying the Max Negative $\times$ Max Negative corner case.

optimal stage depth, the methodologies differ fundamentally in gate utilization. Wallace trees compress bits aggressively at every opportunity, utilizing more 3:2 and 2:2 compressors than strictly necessary. In contrast, the implemented Booth-Dadda tree delays compression until it exceeds the strict mathematical threshold sequence. This approach achieves the identical 4-stage critical path while significantly reducing the total number of instantiated half and full adders, directly minimizing the active silicon area and routing complexity compared to a Wallace equivalent.

Table III. Theoretical Comparison of 16×16 Multiplier Architectures

Architecture	Initial PP Rows	Peak Height	Reduction Stages
Standard Array	16	16	15
Standard Wallace Tree	16	16	6
Standard Dadda Tree	16	16	6
Booth-Wallace (Radix-4)	8	9	4
Booth-Dadda (Implemented)	**8**	**9**	**4**

VII. Conclusion and Future Scope

This project successfully designed, implemented, and verified a high-speed 16×16 signed multiplier at the primitive gate level. By integrating Radix-4 Booth encoding, the architecture successfully halved the partial product rows from 16 to 8. The application of the Modified Sign Extension (MSE) algorithm eradicated the redundant sign-extension triangle, effectively limiting the matrix peak height to 9 bits. Consequently, the combinational Dadda reduction tree optimally compressed the matrix into two 32-bit vectors in exactly four parallel stages utilizing a minimal inventory of 119 full and half adders. Functional verification confirmed the architecture accurately computes two's complement arithmetic across all critical corner cases.

Future optimizations of this architecture will focus on the final carry-propagate addition stage. Replacing the standard Logisim native adder with a parallel-prefix adder, such as a Kogge-Stone or Brent-Kung architecture, would further minimize the delay of the final 32-bit vector merge. Additionally, migrating this structural schematic into a Hardware Description Language (HDL) such as Verilog would allow for direct ASIC/FPGA synthesis, enabling precise sub-nanosecond timing analysis, dynamic power estimation, and automated standard-cell layout generation.

References

[1] C. S. Wallace, "A suggestion for a fast multiplier," *IEEE Transactions on Electronic Computers*, vol. EC-13, no. 1, pp. 14–17, Feb. 1964.
[2] L. Dadda, "Some schemes for parallel multipliers," *Alta Frequenza*, vol. 34, pp. 349–356, 1965.
[3] A. D. Booth, "A signed binary multiplication technique," *The Quarterly Journal of Mechanics and Applied Mathematics*, vol. 4, no. 2, pp. 236–240, 1951.
[4] O. L. MacSorley, "High-speed arithmetic in binary computers," *Proceedings of the IRE*, vol. 49, no. 1, pp. 67–91, Jan. 1961.
[5] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Boston, MA, USA: Addison-Wesley, 2010.
[6] S. R. Kuang, J. P. Wang, and C. Y. Guo, "Modified Booth Multipliers With a Regular Partial Product Array," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 56, no. 5, pp. 404–408, May 2009.